Lasso Regression

1. What Is Lasso Regression?

Lasso Regression stands for **Least Absolute Shrinkage and Selection Operator**. It is a supervised machine learning algorithm used for predicting continuous numerical values.

It is based on Linear Regression but adds a regularization term to control model complexity.

For example, suppose we want to predict a student's exam score using these features:

- `x1`: Study hours
- `x2`: Class attendance
- `x3`: Number of social media hours
- `x4`: Number of unrelated activities

Some features may contribute little to predicting the score. Lasso (L1 regularization) can shrink the coefficients of less useful features to exactly zero, effectively removing them from the model.

Why Do We Need Lasso Regression?

A model may perform very well on training data but poorly on unseen data. This is called **overfitting**. As the complexity of the model increases, it tends to overfit with the data.

Lasso addresses this problem by penalizing large coefficients, encouraging a simpler model that may generalize better.

2. Linear Regression vs. Ridge vs. Lasso

Property	Linear Regression	Ridge Regression	Lasso Regression
Main objective	Minimize prediction error	Minimize error + L2 penalty	Minimize error + L1 penalty
Penalty term	None	`lambda * sum(w_j^2)`	`lambda * sum(abs(w_j))`
Shrinks coefficients	Not explicitly	Yes	Yes
Can make coefficients exactly zero?	Not through regularization	Generally not	Yes
Feature selection	No automatic selection	Usually no exact-zero selection	Yes

Remember: Ridge shrinks coefficients; Lasso can shrink coefficients to zero and perform automatic feature selection.

3. Mathematical Foundation

3.1 Linear Regression Equation

For a single feature:

$$\hat{y} = wx + b$$

Where:

- x = input feature
- w = weight or coefficient
- b = bias or intercept
- $\hat{y}$ = predicted value

For multiple features:

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

The model learns the weights and bias from training data.

3.2 Mean Squared Error (MSE)

Linear Regression measures prediction error using MSE which is the cost function for linear regression:

$$J_MSE = (1/m)\ \Sigma(i = 1 \text{ to } m)\ (y_i - \hat{y}_i)^2$$

Here, m is the number of training samples, y_i is the actual value, and $\hat{y}_i$ is the predicted value.

Why square the errors?

- Positive and negative errors do not cancel each other.
- Large errors receive a larger penalty.
- The model has a mathematical objective to minimize.

3.3 Lasso Cost Function

Lasso adds an L1 penalty to the MSE:

$$J(w,b) = (1/m)\ \Sigma(i = 1 \text{ to } m)\ (y_i - \hat{y}_i)^2 + \lambda\ \Sigma(j = 1 \text{ to } n)\ |w_j|$$

The two components are:

1. **MSE:** Measures how far predictions are from actual values.
2. **L1 penalty:** Penalizes large weights and encourages some weights to become zero.

The parameter λ controls the strength of regularization.

- λ = 0: The objective reduces to ordinary MSE.
- Small λ: Relatively weak regularization.
- Large λ: Stronger pressure toward smaller coefficients.

If λ is too large, the model may underfit because important coefficients can also become too small.

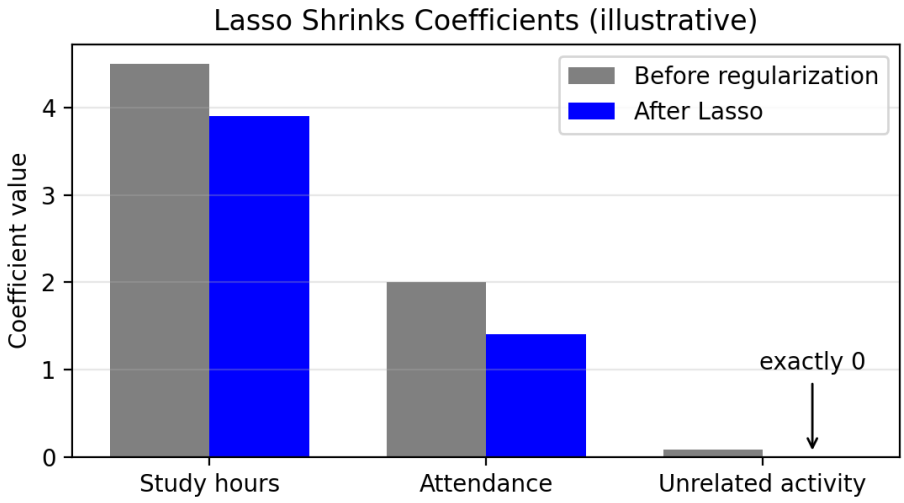
Note: This follows the cost-function convention in these notes. Some implementations use `MSE + lambda * sum(abs(w))`, while others use `0.5 * MSE + lambda * sum(abs(w))`. The scaling convention affects the numerical meaning of λ.

4. Intuition: How Does Lasso Select Features?

Imagine that a model has three features with learned coefficients:

Feature	Before regularization	After Lasso
Study hours	4.5	3.9
Attendance	2.0	1.4
Unrelated activity	0.08	0

The values are illustrative, not results from a real dataset.



The L1 penalty encourages the model to reduce coefficients. In this example, the coefficient of the third feature becomes zero.

The resulting equation becomes:

$$\hat{y} = 3.9x_1 + 1.4x_2 + b$$

The third feature no longer contributes to predictions.

This is the key intuition behind **automatic feature selection**.

5. Gradient Descent for Lasso Regression

Gradient Descent is an optimization algorithm that repeatedly updates weights and bias to reduce the cost function.

The general update rules are:

$$w_j \leftarrow w_j - \alpha \, \partial J/\partial w_j$$

$$b \leftarrow b - \alpha \, \partial J/\partial b$$

Where:

- α = learning rate
- $\partial J/\partial w_j$ = gradient of the cost with respect to a weight
- $\partial J/\partial b$ = gradient with respect to bias

5.1 Gradient of the MSE Component

Using the cost convention $J_{MSE} = (1/m) \Sigma (y_i - \hat{y}_i)^2$:

$$\partial J_{MSE}/\partial w_j = -(2/m) \Sigma(i = 1 \text{ to } m) \, x_{ij}(y_i - \hat{y}_i)$$

For the bias:

$$\partial J_{MSE}/\partial b = -(2/m) \Sigma(i = 1 \text{ to } m) \, (y_i - \hat{y}_i)$$

5.2 Gradient of the L1 Penalty

The penalty for one weight is:

$$\lambda |w_j|$$

Its derivative is:

$$d/dw_j \, |w_j| = +1, \; w_j > 0;$$

$$= -1, \; w_j < 0$$

Therefore, for nonzero weights:

$$\partial J/\partial w_j = -(2/m) \Sigma(i = 1 \text{ to } m) \, x_{ij}(y_i - \hat{y}_i) + \lambda \, \text{sign}(w_j)$$

The corresponding weight update is:

$$w_j \leftarrow w_j - \alpha \, [-(2/m) \Sigma(i = 1 \text{ to } m) \, x_{ij}(y_i - \hat{y}_i) + \lambda \, \text{sign}(w_j)]$$

The bias is usually not penalized, so its gradient remains the MSE gradient.

Important mathematical detail: The absolute-value function is not differentiable at $w_j = 0$. At that point, the implementation can use a subgradient, or a more specialized optimization method can handle the L1 penalty. A simple gradient-descent implementation that uses `np.sign(w)` chooses zero as the subgradient at zero.

6. Building Lasso Regression from Scratch Using NumPy

The following implementation uses batch gradient descent and the cost-function convention explained above. It is a learning implementation rather than a replacement for a production Lasso solver.

Step 1: Complete Code

```
import numpy as np
```

```
class LassoRegression:
    def __init__(self, learning_rate=0.01, n_iterations=1000, lambda_param=0.1):
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.lambda_param = lambda_param

        self.weights = None
        self.bias = None

    def fit(self, X, y):
        # Ensure X and y are NumPy arrays
        X = np.asarray(X, dtype=float)
        y = np.asarray(y, dtype=float).reshape(-1)

        # Number of samples and features
        m, n = X.shape

        # Initialize parameters
        self.weights = np.zeros(n)
        self.bias = 0.0

        # Gradient descent
        for _ in range(self.n_iterations):
            # Predictions
            y_pred = X @ self.weights + self.bias

            # Prediction errors
            error = y_pred - y

            # MSE gradients
            dw = (2 / m) * (X.T @ error)

            # Add L1 regularization gradient
            dw += self.lambda_param * np.sign(self.weights)

            # Bias is not regularized
            db = (2 / m) * np.sum(error)

            # Update parameters
            self.weights -= self.learning_rate * dw
            self.bias -= self.learning_rate * db

        return self

    def predict(self, X):
        X = np.asarray(X, dtype=float)
        return X @ self.weights + self.bias


# Example dataset: two input features
X = np.array([
    [1, 2],
    [2, 1],
    [3, 4],
    [4, 3],
```

```
    [5, 5],
    [6, 4]
], dtype=float)

y = np.array([5, 6, 11, 12, 16, 17], dtype=float)

# Create and train the model
model = LassoRegression(
    learning_rate=0.01,
    n_iterations=5000,
    lambda_param=0.1
)

model.fit(X, y)

# Make predictions
predictions = model.predict(X)

print("Weights:", model.weights)
print("Bias:", model.bias)
print("Predictions:", predictions)
```

Step 2: Understand the Class

Component	Purpose
`__init__()`	Stores the learning rate, iteration count, and regularization strength
`fit()`	Learns weights and bias from training data
`np.zeros(n)`	Initializes all weights to zero
`X @ self.weights + self.bias`	Calculates predictions
`dw += lambda_param * np.sign(self.weights)`	Adds the L1 regularization gradient
`self.weights -= learning_rate * dw`	Updates weights using Gradient Descent
`predict()`	Uses the learned parameters for predictions

Step 3: What Should You Observe?

When you run the code, it prints the learned weights, bias, and predictions.

- Weights change over the iterations.
- Predictions should generally move toward the training targets as the model optimizes the objective.
- Increasing `lambda_param` strengthens the regularization pressure.
- A coefficient may become exactly zero, but this is not guaranteed for every dataset or hyperparameter setting.

To evaluate generalization, use separate training and test datasets rather than relying only on predictions for the data used to fit the model.

7. Practical Considerations

Feature scaling: Lasso is sensitive to feature scales because the L1 penalty acts directly on coefficients. Standardizing numerical input features is generally important before training.

Choosing lambda: Test several regularization strengths using validation data or cross-validation. Too little regularization may leave unnecessary coefficients; too much may remove useful information.

Bias: The implementation does not penalize the bias term. This is the usual convention.

Feature selection does not guarantee better predictions: Removing irrelevant features can simplify a model, but Lasso may also shrink useful coefficients. Evaluate the result on unseen data.

8. Quick Revision Sheet

Question	Short Answer
What is Lasso Regression?	Linear Regression with L1 regularization
Why is regularization used?	To control model complexity and reduce overfitting
What is the L1 penalty?	λ Σ(j) |w_j|
What does lambda control?	Regularization strength
What is Lasso's special advantage?	It can make coefficients exactly zero
What is automatic feature selection?	Features with zero coefficients stop contributing to predictions
What is Gradient Descent?	An iterative optimization algorithm
Is the bias penalized here?	No
What happens if lambda is too large?	The model may underfit
Why scale features?	To make coefficient penalties more comparable across features

9. Test Yourself

Try answering these without looking at the notes.

1. Why can Lasso Regression perform feature selection while Ridge Regression generally does not?
2. What happens to the cost function when λ = 0?
3. Why does the L1 penalty use the absolute value of each weight?
4. What is the role of `np.sign(self.weights)` in the code?
5. Why should the bias usually remain unregularized?
6. If lambda becomes very large, what problem might occur?
7. Why should we evaluate a Lasso model on unseen data?